

ETH Quantum Hackathon 2026

QuEra's Challenge

Quantum Icing

Alessandra Iula Federico Quartieri Alessandro Ruzza Daniele Salvi

Single-qubit rotation synthesis

Resource costs for $R_z(\pi/2^n)$ under fault-tolerant constraints

Main claim

A single-qubit rotation can require substantial overhead once non-Clifford synthesis, injection protocols, and logical encoding are included.

Gates synthesis

Target family

$$R_z(\pi/2^n), \quad n \in \{0, 1, 2, 3, 4, 5, \dots\}$$

We compared an implementation V with the target U using the challenge distance

$$d(U, V) = \sqrt{1 - \frac{|\text{Tr}(U^\dagger V)|}{2}}.$$

Technical question

On paper, R_z acts on one qubit.

Under fault-tolerant constraints, the non-Clifford component can dominate the resource cost.

Gate set

$$\{H, S, T, \text{CNOT}\}$$

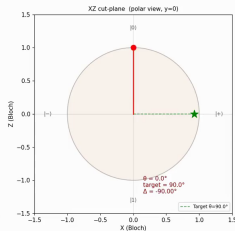
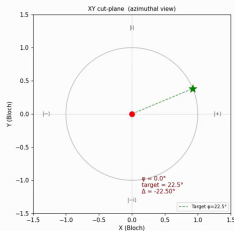
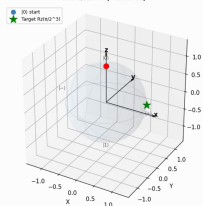
Synthesis of $R_z(\pi/2^n)$

```
def Rz_gate(n, epsilon=1e-10):  
    if n == 0: return X_gate  
    if n == 1: return S_kernel  
    if n == 2: return T_kernel  
  
    theta = np.pi / 2**n  
    qk_circ = gridsynth_rz(theta, epsilon)  
    sequence = gate_sequence_from_circuit(qk_circ)  
    return load_circuit_as_squin_kernel(sequence)
```

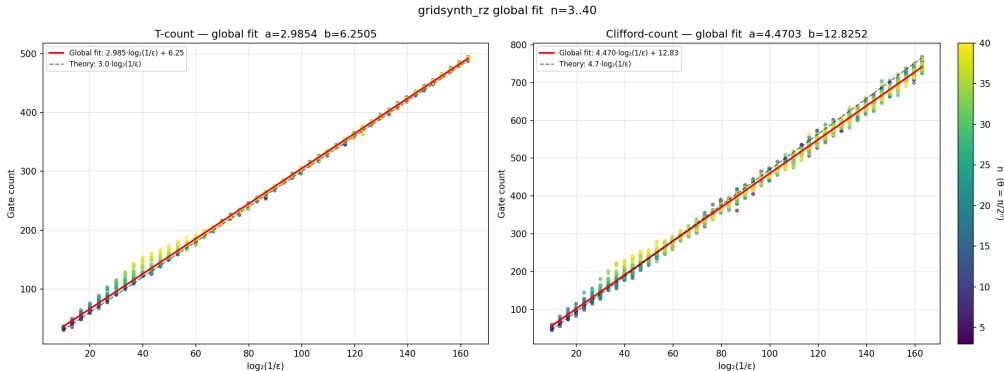
Strategy

- Exact special cases first.
- For $n \geq 3$: use gridsynth_rz.
- Convert Qiskit circuit into a one-qubit Squin kernel.

Part 2 - $R_z(\pi/2^3)$ synthesis via Clifford+T
Gate 1/126: H [T-count=50]

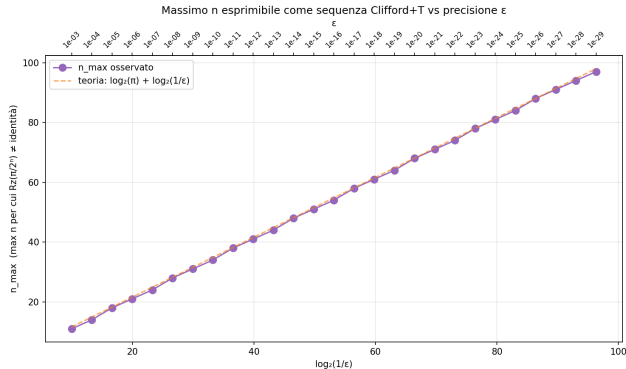


Synthesis scaling



Across the grid, the fitted scaling is close to the expected synthesis law: T -count grows roughly linearly in $\log_2(1/\epsilon)$, with slope near 3 in our data. We also found out that the Clifford gates grow with slope near 4.5

Arbitrarily small rotations require sufficiently small ϵ



Interpretation

- The observed threshold follows the theoretical trend

$$n_{\max} \approx \log_2(\pi) + \log_2(1/\epsilon).$$

- Therefore, in principle, we can approximate rotations that are *as small as we want*, provided that ϵ is chosen sufficiently small.
- The trade-off is cost: decreasing ϵ increases the Clifford+ T synthesis overhead.

Why fault tolerance changes the cost of a 1-qubit gate

Transversal logical gate

- Implemented by independent physical gates across the code block.

- Typical form:

$$U_L = U^{\otimes n}$$

or pairwise CNOTs between encoded blocks.

- A single fault does not spread inside the same block.

Non-transversal logical gate

- Cannot be implemented as one independent physical layer.
- A direct implementation may spread errors or leave the code space.
- Requires a gadget: injection, distillation, teleportation, code switching.

Main point for the challenge

Physical level

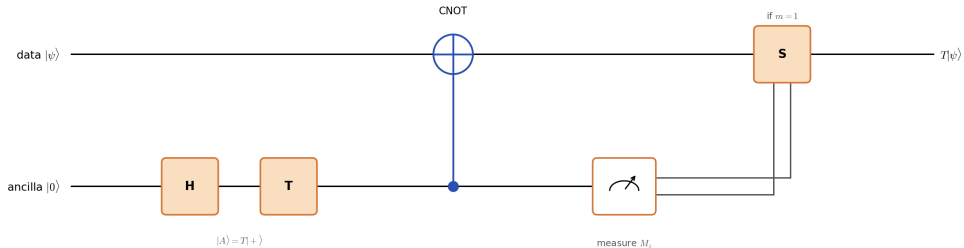
The T gate is a single-qubit operation. If it is available directly, it can simply appear as one gate inside a Clifford+ T synthesis sequence.

Logical fault-tolerant level

In many quantum error-correcting codes, including the Steane code and the surface code, the T gate is **not transversal**. Therefore it cannot be applied as a simple physical layer across the encoded block.

This is where a single-qubit gate becomes expensive.

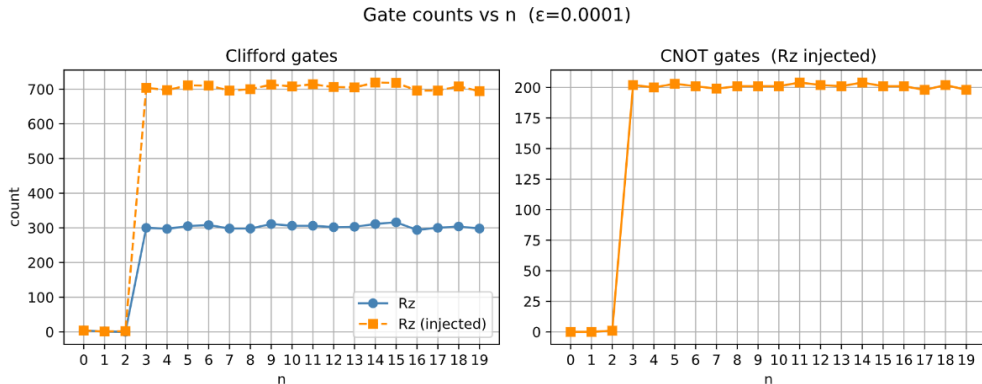
Part 3: when T is not available on the data qubit



Per injected T or $T^\dagger$

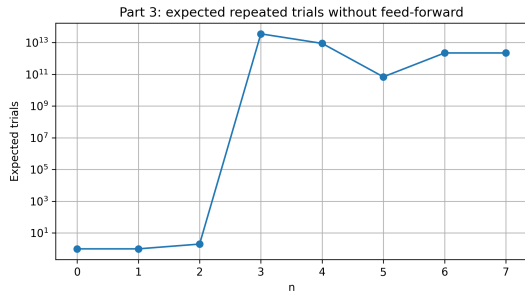
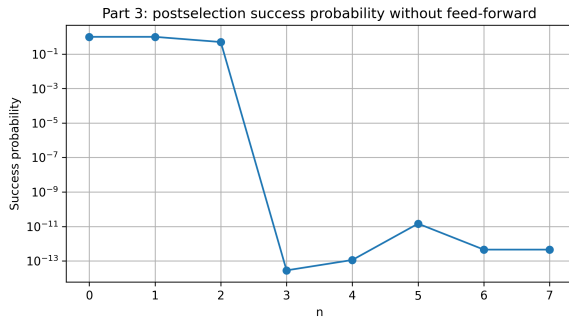
1 magic-state ancilla + 1 CNOT + 1 measurement + 1 feed-forward correction

Part 3 result: the T -count becomes a resource multiplier



Direct synthesis produces a Clifford+ T word. Injection replaces every non-Clifford gate with an ancilla-assisted protocol, so each $T/T^\dagger$ contributes additional operational overhead.

Part 3 without feed-forward: postselection cost



Rule of thumb

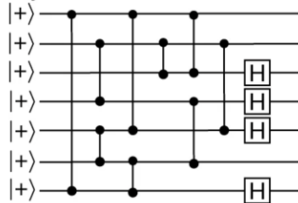
If each T injection succeeds under postselection with probability $1/2$, a sequence with m injected $T/T^\dagger$ gates succeeds with probability 2^{-m} .

Part 4: one physical qubit becomes one logical qubit

Encoding circuit



Physical

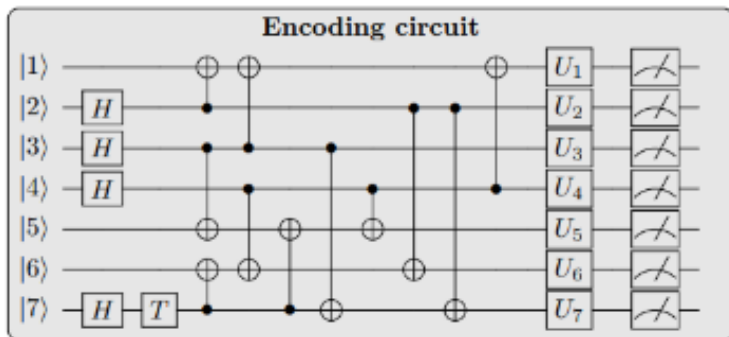


Steane $[[7, 1, 3]]$ implementation

- Prepared logical $|0_L\rangle$ with the graph-state circuit.
- Used transversal logical H , S , and CNOT.
- Treated T as non-transversal, hence supplied by logical magic-state injection.

Now one logical T injection requires an additional 7-qubit logical magic block, plus logical CNOT and logical measurement.

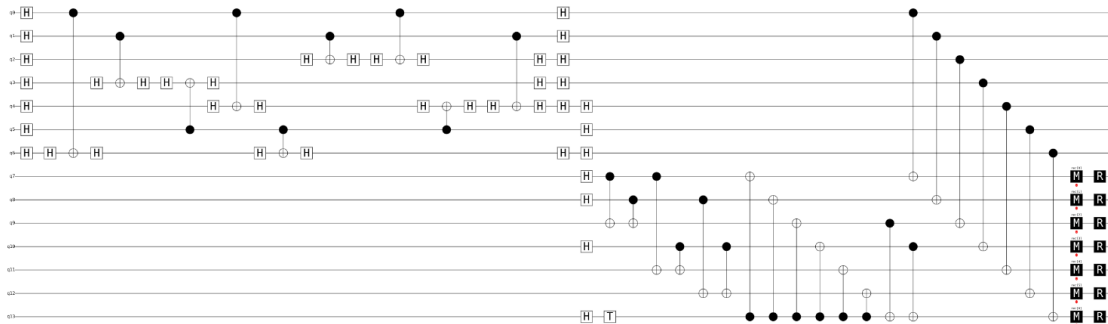
Part 4: Magic state representation from the literature



Reference

The figure is taken from Bluvstein et al., *Logical quantum processor based on reconfigurable atom arrays*, arXiv:2107.07505.

Part 4: logical CNOT implementation



Interpretation

The logical CNOT is implemented transversally across the Steane code blocks, which is one of the main reasons why Clifford operations remain comparatively manageable at the logical level.

Part 4: logical T injection in the code

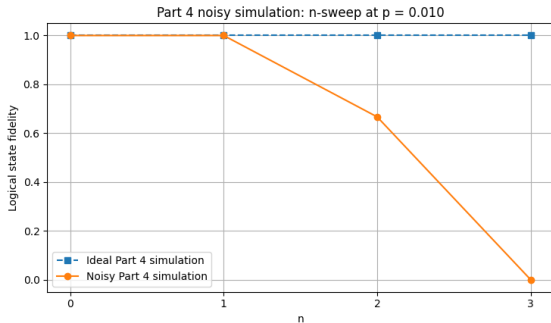
```
def Steane_T_injection_logical(data, magic):  
    Steane_CNOT_logical(data, magic)  
    m = Steane_measure_logical_Z_weight3(magic)  
    if m:  
        Steane_S_logical(data)  
    reset(magic)  
    return data
```

Simulation choice

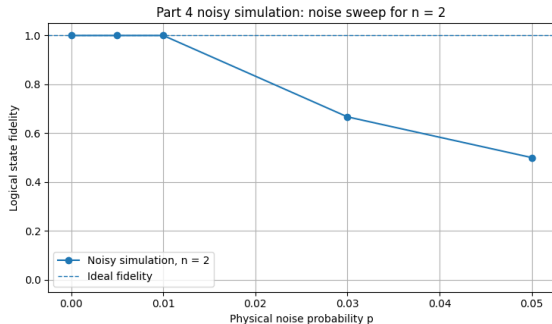
- Use 7 data qubits + 7 magic qubits.
- Reuse one magic block by preparing it before each injection.
- Compare the reduced density matrix of the data block with the ideal logical state.

This keeps the simulation tractable: many logical T gates are treated by reusing the same 14-qubit workspace instead of allocating one new block per T .

Part 4: noisy logical simulation



Fixed physical noise $p = 0.010$: fidelity drops as the synthesized rotation requires more injected logical T gates.

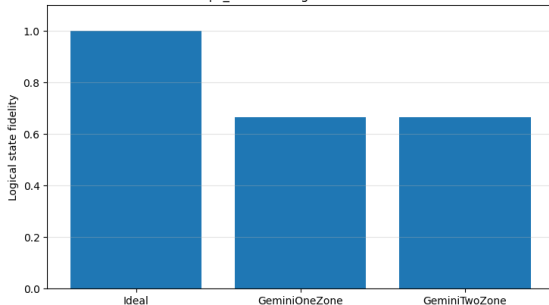


Fixed target $n = 2$: increasing physical noise reduces the logical state fidelity after Steane-level T injection.

Takeaway: the logical construction works ideally, but its reliability becomes sensitive to the accumulated physical noise.

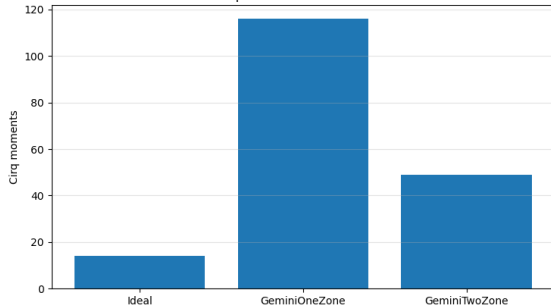
Part 4: Gemini noise models

Part 4: Steane $|+_L\rangle$ encoding under Gemini noise models



Fidelity comparison after applying Bloqade Gemini noise models to the emitted Steane encoding circuit.

Part 4: circuit depth after Gemini noise transformation

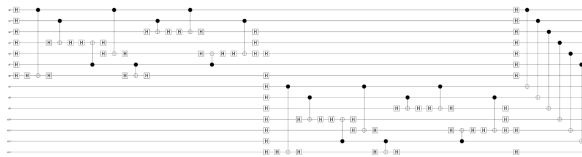
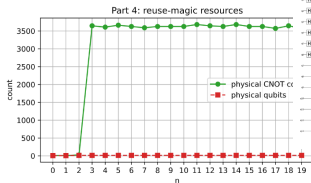
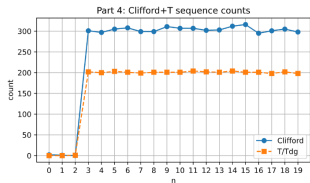


Circuit-depth comparison: the noise transformation increases the number of Cirq moments, especially in the one-zone model.

Takeaway: hardware-aware noise models expose both fidelity loss and temporal overhead beyond the ideal logical circuit.

Part 4 metrics and Bell-state circuit

Part 4 — reuse-magic gate counts ($\epsilon=1e-20$)

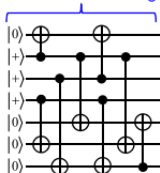


Gate count: CNOT: 25

H: 65

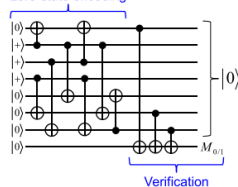
Fault-tolerant encoding

Non-fault-tolerant
zero-state encoding



Non-fault-tolerant encoded-state preparation

Non-fault-tolerant
zero-state encoding

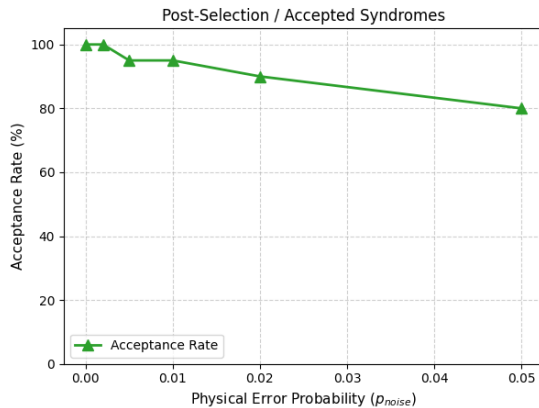
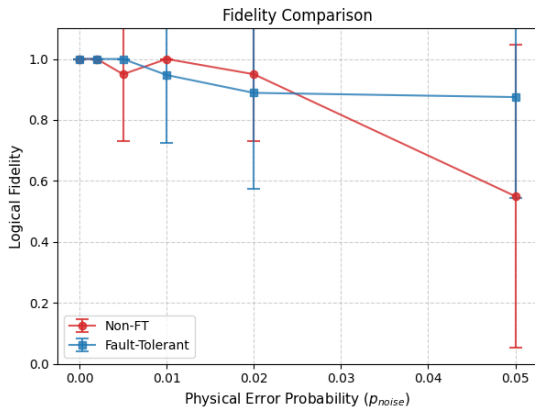


Fault-tolerant preparation with verification

Version	Pros	Cons
Non-fault-tolerant encoding	Low qubit overhead; short circuit; useful for ideal simulation and resource counting.	A single physical fault during preparation can propagate into a multi-qubit or logical error.
Fault-tolerant encoding	Uses verification/postselection or error detection; suppresses logical errors to higher order in the physical error rate.	Requires extra ancillas, measurements, repetitions, and larger circuit depth.

Fault-Tolerant vs Non-Fault-Tolerant Under Noise

Performance Analysis: Fault-Tolerant vs Non-Fault-Tolerant



Team STAR

Transversal R_Z on the surface code

Space-Time Analog Rotation (Akahoshi et al., 2023)

Core question

T is not transversal on the surface code. Can we still implement an *arbitrary-angle* logical $R_Z(\theta_L)$ without a magic-state factory?

STAR: distilling a logical rotation from d physical ones

Protocol

- 1 Initialize the logical state
- 2 2 rounds of syndrome extraction
- 3 Apply $R_Z(\theta_{\text{phys}})$ **transversally** to the first column of d data qubits.
- 4 Run d syndrome rounds — postselect || decode.
- 5 Logical unrotation: CNOT chain + $R_Z(-\theta_L)$.

Postselect on errors before the transversal physical R_Z .

$d = (3, 5)$ feasible

$d = 7$ slow

Angle relation

$$\theta_{\text{phys}} = 2 \arctan\left(\tan \frac{\theta_L}{2}\right)^{1/d}$$

For small θ_L :

$$\theta_{\text{phys}} \approx \theta_L^{1/d} \gg \theta_L$$

A modular Squin reconstruction: one factory, all experiments

```
# One factory -- parameterized by d and angle
init_q, syn_round, phys_rot = make_helpers(d, phys_angle)

@squin.kernel
def star_circuit():
    data      = squin.qalloc(num_data)
    ancillas = squin.qalloc(num_ancillas)
    init_q(data, ancillas)    # project into code space
    m0 = syn_round(data, ancillas)
    m1 = syn_round(data, ancillas)

    # detectors on m0, m1
    for _ in range(d):
        syn_round(data, ancillas)

    phys_rot(data)            # transversal Rz on col 0
    # unrotation, readout...
```

Three kernel primitives

Kernel	Role
init_q	Reset + project into code space
syn_round	One Z+X syndromes
phys_rot	Transversal R_Z on first column

`make_helpers_noisy(d, a, p1, p2)` adds depolarizing noise — swap noise in/out with one call, same structure throughout.

Kernels are first-class Squin objects — not plain Python functions. Tsim traces, compiles, and exports them to Stim automatically. The same three primitives drive all three parts of the STAR challenge.

A modular Squin reconstruction: one factory, all experiments

```
def make_helpers(d, angle):
    z_supports, x_supports = build_stabilizer_supports(d)
    num_data = d**2
    num_z, num_x = len(z_supports), len(x_supports)
    num_ancillas = num_z + num_x

    @squin.kernel
    def init_q(data: Register, ancillas: Register) ->
        Measurement:
            # initialize state, discard z-syndrome

    @squin.kernel # must be a kernel, not plain fn
    def syn_round(data: Register, ancillas: Register) ->
        Measurement:
            # Reset and do CX ancilla-data
            # Loop on z_supports and x_supports
            return squin.broadcast.measure(ancillas)

    @squin.kernel
    def phys_rot(data: Register):
        for idx in range(0, num_data, d):
            squin.rz(angle, data[idx])

    return init_q, syn_round, phys_rot
```

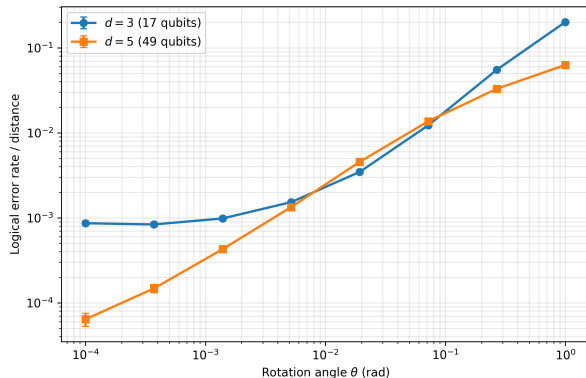
Three kernel primitives

Kernel	Role
init_q	Reset + project into code space
syn_round	One Z+X syndromes
phys_rot	Transversal R_Z on first column

`make_helpers_noisy(d, a, p1, p2)` adds depolarizing noise — swap noise in/out with one call, same structure throughout.

Kernels are not plain Python functions. Tsim traces, compiles, and exports them to Stim.

Part 2: fidelity vs. rotation angle under depolarizing noise



Plot axes

- x : rotation angle θ (log).
- y : p_L/d — logical errors per accepted shot, normalised by d (log).
- Curves for $d = 3, 5, 7$ at $p_{1,2} = 10^{-3}$.

Observations

- Small θ : low error — rotation barely perturbs the code space.
- Large θ : error rises with noise.
- $+2$ in $d \approx$ one decade reduction in p_L/d .

Postselection and adaptive sampling

Why postselect?

STAR is *partially* fault-tolerant — not fully. Any syndrome firing in the pre-rotation rounds signals a physical error; those shots are discarded.

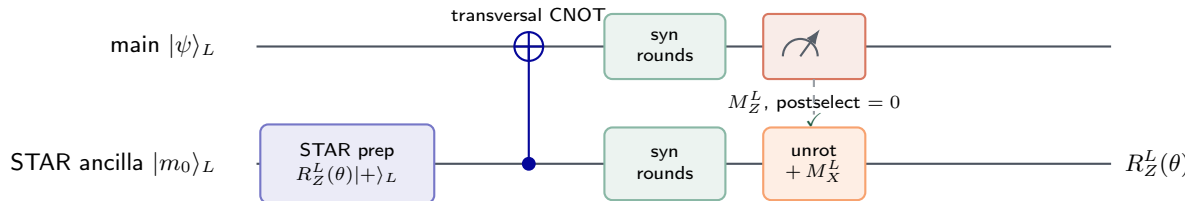
$$\frac{p_L}{d} = \frac{\text{logical errors}}{\text{accepted shots} \times d}$$

Conservative postselection reduces acceptance rate.

Why use the decoder for postselection?

- Raw check ($\forall i : d_i = 0$) rejects *any* triggered detector — including correctable errors.
- MWPM infers the most likely error chain; we accept a shot iff it predicts no logical correction ($\hat{e}_L = 0$).
- Result: correctable syndromes are retained, acceptance rate improves significantly.

Part 3: teleporting the rotation into the logical qubit



Two independent $[[d^2, 1, d]]$ blocks — four registers

`data_m, stab_m (main) + data_s, stab_s (ancilla)`

Transversal CNOT

```
for i: squin.cx(data_s[i],  
data_m[i])
```

One physical gate per data qubit. Genuinely transversal.

Disclosure slide

AI assistance was used as a learning tool to explore unfamiliar concepts and techniques. It helped us quickly get up to speed in areas outside our prior experience, allowing us to spend more time on deeper understanding rather than initial groundwork.

This slide is included because the challenge asks for a brief AI-use disclosure in the final presentation when AI tools are used.

Scan the QR Code



Open the interactive Bloch sphere demo

<https://say-ensures-thru-bench.trycloudflare.com/>